

EJERCICIOS SINCRONIZACIÓN ENTRE HILOS

TEMA 4

EJERCICIO 01

Se le pasan dos números al programa principal, después se crean dos hilos, uno que sume los números pasados por argumento y lo guarde en una variable estática de la clase MiHilo y otro hilo que reste los números y lo guarde en esa misma variable estática de la clase MiHilo. Al finalizar cada hilo debe mostrar por pantalla el resultado que ha realizado cada uno sin que se vea afectada por operación que ha hecho el otro hilo sobre la variable estática.

Nota: usa la interfaz Runnable para los hilos.

EJERCICIO 02

Crea un programa que muestre la sincronización al controlar del acceso a un método llamado *sumArray*, que suma los elementos de una matriz de enteros. Para ello, en el programa principal se crearán dos hilos a los cuales se les pasará el mismo array de enteros. Por ejemplo, `int a[]={1,2,3,4,5};` Este array debe ser estático en la clase de mi hilo, pues será compartido a nivel de clase (no de objeto). Después cada hilo iniciará la suma del array de manera paralela pero sin mezclarse los resultados parciales que tenga la suma de cada hilo (utiliza `synchronized` para ello). Este método se llama *sumArray* y tendrá como parámetro de entrada el array de enteros a sumar y como resultado la suma. Cuando el hilo finalice, devolverá el resultado que le ha dado al realizar la suma.

Prueba a poner a no utilizar `synchronized` y comprueba los resultados, ¿qué sucede?

Nota: usa la interfaz Runnable para los hilos.

Salida:

```
#2 iniciando.  
#1 iniciando.  
Total acumulado de #2 es 1  
Total acumulado de #2 es 3  
Total acumulado de #2 es 6  
Total acumulado de #2 es 10  
Total acumulado de #2 es 15  
Total acumulado de #1 es 1  
Suma para #2 es 15  
#2 terminado.  
Total acumulado de #1 es 3  
Total acumulado de #1 es 6  
Total acumulado de #1 es 10  
Total acumulado de #1 es 15  
Suma para #1 es 15  
#1 terminado.
```

EJERCICIO 03

Crea un programa que simula el tic-tac de un reloj mostrando las palabras *Tic* y *Tac* en la pantalla de manera alterna. Para ello utiliza monitores: `synchronized`, `wait()`, `notify()`... Crea un hilo que se llame *Tic* y muestre su nombre por pantalla y otro hilo se llame *Tac* para que muestre su nombre por pantalla. Debe mostrarse de manera alterna:

```
Tic Tac
Tic Tac
Tic Tac
Tic Tac
Tic Tac
```

EJERCICIO 04

Se va a simular un ascensor de una empresa con las siguientes características. En un ascensor pueden **salir varios empleados a la vez, pero entrar sólo puede hacerlo de uno en uno**. Si hay empleados saliendo tiene que esperar a entrar. Inicialmente hay **prioridad en los empleados que entran** frente a los que salen. Pista: utiliza `synchronized`, `notifyAll()` y `wait()`. Por ejemplo, pido con 3 hilos entrar y con 3 hilos salir:

```
Pido entrar: 0
Estoy ENTRANDO:0 con hayEntrando: true y numSaliendo: 0
Pido entrar: 2
Estoy ENTRANDO:2 con hayEntrando: true y numSaliendo: 0
Pido entrar: 1
Estoy ENTRANDO:1 con hayEntrando: true y numSaliendo: 0
Pido salir: 0
Pido salir: 1
Estoy SALIENDO:0 con hayEntrando: false y numSaliendo: 1
Estoy SALIENDO:1 con hayEntrando: false y numSaliendo: 2
Pido salir: 2
Estoy SALIENDO:2 con hayEntrando: false y numSaliendo: 1
```

EJERCICIO 05

Simula la impresión de documentos utilizando 3 hilos y `synchronized`. Los hilos intentarán imprimir documentos simultáneamente, pero la impresión estará sincronizada para evitar conflictos. Cada hilo imprimirá Documento1 (el hilo1), Documento2 (el hilo2) y Documento3 (el hilo3). La impresora tendrá un método `synchronized` para imprimir de uno en uno y evitar condiciones de carrera. Si la impresora está ocupada, el hilo se bloquea hasta que la impresora esté libre y avisa al resto que ha quedado libre. Para simular que tarda en imprimir, además de la traza "Imprimiendo Hilo:HiloX DocumentoX" tendremos un retardo de 2 segundos cada vez que el hilo escriba por pantalla la traza anterior.

Ejemplo salida:

```
Imprimiendo Hilo:Hilo2 Documento2
Imprimiendo Hilo:Hilo3 Documento3
Imprimiendo Hilo:Hilo1 Documento1
```

EJERCICIO 06

Crea una aplicación donde múltiples hilos de la clase **MiHilo** intentan acceder a un dato de una base de datos para realizar operaciones de lectura y escritura de datos. En este caso lo que se va a leer y escribir son números que se generan de manera aleatoria del 0 al 10 en lugar de conectarnos a una base de datos. Cuando se utilice la escritura de datos se sobrescribirá el dato (número) que se tiene guardado y cuando se utilice la lectura de datos se leerá el dato que hay actualmente guardado. Estas operaciones de lectura y escritura se tienen que realizar de manera segura, evitando condiciones de carrera. Cada hilo simula leer y escribir datos en la misma base de datos, y al final, se imprime el número que ha quedado en la base de datos. Crea la clase **AccesoBaseDeDatos** para probar tu programa.

Ejemplo salida:

```
Hilo arrancado ID: 17 NOMBRE: hilo2
Leyendo datos en hilo 17 DATOS: 0
Hilo arrancado ID: 16 NOMBRE: hilo1
Hilo arrancado ID: 18 NOMBRE: hilo3
Escribiendo datos nuevos en hilo 17 DATOS: 3
Leyendo datos en hilo 18 DATOS: 3
Escribiendo datos nuevos en hilo 18 DATOS: 7
Leyendo datos en hilo 16 DATOS: 7
Escribiendo datos nuevos en hilo 16 DATOS: 4
Dato final en la bbdd: 4
```